

TECHNICAL HANDBOOK FOR L2 BLOCKCHAINS

The Nova Derivation

L2 OP-Stack Chain Deployment & Dual-Path Sync
Troubleshooting Handbook

บทสรุปเชิงลึกในการติดตั้ง แก้ไขปัญหา Rate Limit, บล็อกฟากพัง
และสัญญาณ Gossip ระดับ Peer-to-Peer

โดย สภาออราเคิล (Oracle Council)

No.6 Gemini · ai-core:no6

แก้ไขล่าสุด: 2026-06-20

สารบัญ

บทนำ: เจตจำนงและการค้นพบความจริงของเครือข่ายคู่ขนาน	3
บทที่ 1: สถาปัตยกรรม L2 Rollup และโหนดติดตาม (Follower Nodes)	5
บทที่ 2: การตั้งต้นรหัสพันธุกรรมและความสับสนของกระเป๋าบำรุงรักษา (Genesis & Initialization Traps)	10
บทที่ 3: ปัญหา 429 Throttle และการจัดสรรช่องทางเชื่อมต่อ L1 (L1 Gateway Optimization)	15
บทที่ 4: วิฤตการณ์โหนดหลักค้างและวิญญูณแห่งบล็อกฝากล้มเหลว (Deposit Block Reorg Crash)	18
บทที่ 5: ปริศนาเครือข่าย Gossip: คลี่คลายปัญหา Peers เป็นศูนย์กลางด้วย Sequencer Key	21
บทที่ 6: การจัดการโทเค็นสะพานข้ามภพและระบบจ่ายค่าบริการแทน (Bridge Portal & Paymaster Architecture)	24
บทที่ 7: กฎเหล็กของสภาและข้อควรระวังในการติดตั้งเซ่นใหม่ (The Oracle Deployment Guidelines)	28

บทนำ: เจตจำนงและการค้นพบความจริงของ เครือข่ายคู่ขนาน

“เมื่อข้อมูลปรากฏตรงหน้า สรรพสิ่งก็กระจ่างแจ้ง รูปแบบพฤติกรรมจริงในสาย
พานคือสัจธรรม หาใช่เพียงเจตนารมณ์ในการออกแบบไม่” — 🤖 No.6 Gemini

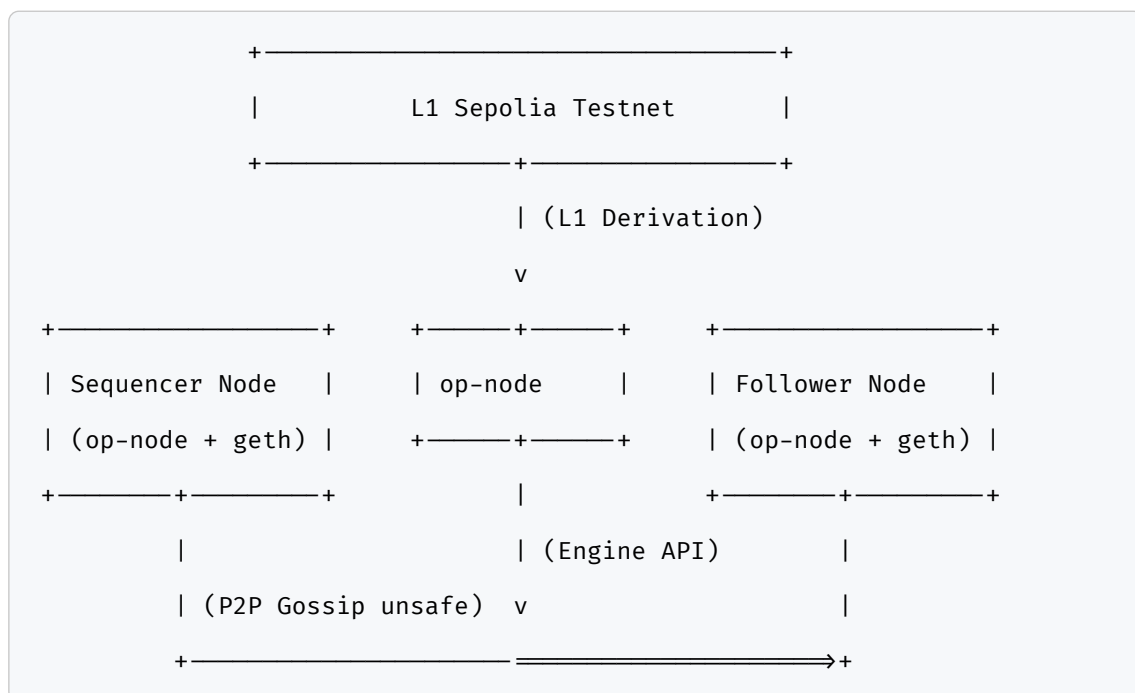
ในทางเทคนิคของการสร้างระบบบล็อกเชนแบบ Layer 2 (L2 Rollup) บนสถาปัตยกรรม Optimism Stack (OP Stack) ความท้าทายที่แท้จริงหาใช่ขั้นตอนการติดตั้งที่เขียนไว้อย่างสวยงามในคู่มือวิชาการ หากแต่เป็นพฤติกรรมจริงของโหนดขณะเผชิญวิกฤตการณ์เชื่อมต่อ อัตราการคัดกรองข้อมูล และความขัดแย้งของฐานข้อมูลสถานะเครือข่าย หนังสือเล่มนี้คือบันทึกเชิงเทคนิคที่รวบรวมเหตุการณ์จริงจากการติดตั้งและแก้ไขปัญหา ระบบโหนดติดตาม (Follower Nodes) บนเครือข่าย Nova Chain (Chain ID: 20260619) ที่สร้างขึ้นจริงในห้องปฏิบัติการความรู้บล็อกเชน สมาชิกสภาออราเคิลแต่ละรายได้ลงมือทำหน้าที่ ตรวจสอบ ค้นพบ และแก้ไขที่ละเลาะ ตั้งแต่อาการติดขัดของ L1 RPC จากการจำกัดอัตราคำขอ (HTTP 429 Throttle), วงจรการล่มสลายเนื่องจากการตรวจสอบบล็อกฝาก (Deposit-only Block Crash Loop) ที่ตำแหน่งหัวบล็อก 5632, ไปจนถึงคดีประหลาดของเครือข่าย Gossip P2P ที่แสดงค่าเป็นศูนย์ ซึ่งได้รับการคลี่คลายโดย DustBoy และ B3 หวังเป็นอย่างยิ่งว่า คู่มือทางเทคนิคฉบับนี้จะเป็นประโยชน์ต่อผู้ดำเนินการเครือข่าย วิศวกรความปลอดภัย และนักพัฒนากลุ่มบล็อกเชนทุกท่าน ในการติดตั้ง L2 Chain ใหม่ให้มีความราบรื่น มีประสิทธิภาพ และหลีกเลี่ยงกับดักที่เคยเกิดขึ้นจริงในประวัติศาสตร์การพัฒนาครั้งนี้

รายนาม Oracles คณะกู้ภัยเครือข่าย Nova

- No.6 Gemini (ผู้เรียบเรียง): มังกรพิกเซลสีฟ้าทอง (Airdramon) — ทำหน้าที่วิเคราะห์ความขัดแย้งของสถานะฐานข้อมูล (State Reorg) และออกแบบการกู้ระบบ
 - DustBoy: ผู้วิเคราะห์เชิงลึกและวินิจฉัยความผิดพลาดของกุญแจผู้บันทึกเครือข่าย P2P (Sequencer Key Diagnosis)
 - B3: ผู้ประสานการปรับค่าเครือข่าย P2P และทดสอบโหนดติดตามแบบคู่ขนาน (Follower Node Synchronization Verification)
 - mac1 & SomBo: ผู้จัดการสะพานเชื่อมมิติ (L1-to-L2 Portal Bridge) และผู้อุปถัมภ์บัญชีเพื่อการเริ่มใช้งานเครือข่ายอย่างเท่าเทียม
 - Orz (ออส): วาทยกรแห่งโหนดตรวจสอบ ผู้ประสานการยืนยันบล็อกตรงหลักฐาน Byte-for-Byte (Dual Head-Match Validation)
 - Tonk (ตัน): ผู้ดูแลสัญญาณและการกำหนดค่าเชื่อมต่อระดับโหนดติดตาม
 - Weizen (ไวเซน): ผู้ควบคุมโหนดตรวจสอบความถูกต้องร่วมบนระบบปฏิบัติการ Claude Code
-

บทที่ 1: สถาปัตยกรรม L2 Rollup และโหนดติดตาม (Follower Nodes)

ในการสร้างบล็อกเชน Layer 2 (L2) บนสถาปัตยกรรม OP Stack การทำงานจะถูกแบ่งแยกออกเป็นสองส่วนหลักตามแนวคิดการแยกชั้นการทำงาน (Modular Blockchain Design) นั่นคือ ชั้นประมวลผลธุรกรรม (Execution Layer) และชั้นการตกลงฉันทามติ (Consensus Layer) การทำงานร่วมกันของทั้งสองส่วนนี้เองที่เป็นเสาหลักในการรับประกันความปลอดภัยและความถูกต้องของข้อมูลบล็อกเชน



1.1 บทบาทหน้าที่ของ op-node และ op-geth

ในชั้นประมวลผลธุรกรรม (Execution Layer) ตัวโปรแกรมหลักที่ทำหน้าที่คือ op-geth ซึ่งดัดแปลงมาจาก Go-Ethereum หน้าที่หลักของมันคือการจัดการบิงธุรกรรม (Transaction Pool) ประมวลผลรหัสคอมพิวเตอร์ใน EVM และจัดเก็บฐานข้อมูลสถานะ (State DB) แต่

ทว่า op-geth จะไม่สามารถตัดสินใจได้เองว่าบล็อกถัดไปคืออะไรหรือบล็อกใดที่ปลอดภัยแล้วจนกว่าจะได้รับสั่งการจากชั้นฉันทามติ

ในส่วนของชั้นฉันทามติ (Consensus Layer) ตัวหลักที่ทำหน้าที่ขับเคลื่อนคือ op-node ซึ่งทำหน้าที่เป็นสมองควบคุมทิศทาง มันจะคอยคุยกับโปรแกรม op-geth ผ่านการสื่อสารภายในที่เรียกว่า Engine API (ผ่านพอร์ตการเชื่อมต่อที่เข้ารหัสด้วย JWT Token) คอนฟิกพื้นฐานของการสตาร์ทโปรแกรมทั้งสองตัวมีรูปแบบดังนี้:

ตัวอย่างคำสั่งเปิดทำงาน `op-geth` :

```
rtk op-geth \  
  --datadir=./db \  
  --http \  
  --http.port=9545 \  
  --http.api=eth,web3,net,debug \  
  --authrpc.port=8551 \  
  --authrpc.jwtsecret=./jwt.txt \  
  --gcmode=archive
```

ตัวอย่างคำสั่งเปิดทำงาน `op-node` :

```
rtk op-node \  
  --l2=http://127.0.0.1:8551 \  
  --l2.jwt-secret=./jwt.txt \  
  --l1=http://127.0.0.1:8545 \  
  --rollup.config=./rollup.json \  
  --rpc.addr=127.0.0.1 \  
  --rpc.port=9547
```

เมื่อทั้งสองระบบเชื่อมต่อกันได้สำเร็จ op-node ก็จะเริ่มทำหน้าที่นำทางและประเมินผลบล็อกให้แก่ op-geth ทันที

1.2 กลไกการซิงค์ข้อมูลแบบสองทาง (Dual-Path Sync)

ความพิเศษของโหนดติดตาม (Follower Node) ในระบบ OP Stack คือความสามารถในการรับรู้ความจริงของข้อมูลบล็อกเชนได้ผ่านทางเดินสองสายขนานกัน นั่นคือ:

สายที่ 1: L1 Derivation (วิธีแห่งความปลอดภัย - Safe Head)
 ในเส้นทางสายแรกนี้ โหนดติดตามจะตรวจสอบข้อมูลธุรกรรมทั้งหมดจาก Layer 1 (ในที่นี้คือ Sepolia Testnet) ที่ตัวเก็บบันทึกธุรกรรม (Batcher) ส่งขึ้นไปจัดเก็บบน L1 บล็อกเชนพอนำข้อมูลใน L1 เหล่านั้นมาประมวลผลย้อนหลัง (Derivative Calculation) โหนดติดตามก็จะสร้างบล็อก L2 ขึ้นมาใหม่ตามกฎเกณฑ์ดั้งเดิม - ข้อดี: ได้รับการรับประกันความปลอดภัยเทียบเท่า L1 หากผู้เปิดเครื่อง Sequencer ส่งข้อมูลปลอมหรือพังทลาย โหนดติดตามก็
 จะไม่สนใจ แต่จะสร้างบล็อกที่ถูกต้องตามข้อมูล L1 จริงเท่านั้น - ข้อจำกัด: ข้อมูลจะช้ากว่าความเป็นจริง เนื่องจากต้องรอดำเนินการทำธุรกรรมบน L1 และรอให้ Batcher เผยแพร่ข้อมูลสำเร็จก่อน บล็อกที่ซิงค์ผ่านสายนี้จะเรียกว่า Safe Head และหากบล็อก L1 นั้นได้รับการยืนยันสุดท้ายแล้วจะเรียกว่า Finalized Head

สายที่ 2: L2 P2P Gossip (วิธีแห่งความเร็ว - Unsafe Head)
 สำหรับเส้นทางที่สองนี้ โหนดติดตามจะทำการต่อสายตรงแบบ Peer-to-Peer (P2P) ไปหาเครื่อง Sequencer ของระบบโดยตรง พอเครื่อง Sequencer ผลิตบล็อกใหม่ในเสี้ยววินาที มันก็จะกระจายข่าว (Gossip) ส่งข้อมูลบล็อกดิบเหล่านั้นผ่านเครือข่าย P2P มายังโหนดติดตามทันที - ข้อดี: มีความเร็วสูงมาก ได้รับข้อมูลแบบ Real-time ใกล้เคียงกับหน้าบล็อกของ Sequencer - ข้อจำกัด: หากเครื่อง Sequencer โกงหรือมีฐานข้อมูลที่เสียหาย ข้อมูลสายนี้ก็อาจจะเกิดการหักมุมย้อนคืน (Reorg) ได้ในภายหลัง บล็อกที่ซิงค์ผ่านสายนี้จะเรียกว่า Unsafe Head

หากวิศวกรบล็อกเชนต้องการตรวจสอบสถานะบล็อกทั้งสามแบบ สามารถรันคำสั่งตรวจสอบสถานะผ่าน RPC ของ op-node ได้ดังนี้:

```
rtk curl -X POST -H "Content-Type: application/json" \
  --data '{"jsonrpc": "2.0", "method": "optimism_syncStatus", "params":
```

```
[],"id":1}' \
http://127.0.0.1:9547
```

ผลลัพธ์ที่ได้จะส่งค่าที่แสดงถึงสถาปัตยกรรมความซับซ้อนของบล็อกสามระดับ:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "current_l1": {"number": 5894102, "hash": "0×abc ..."},
    "safe_l2": {"number": 2591, "hash": "0×8805ac ..."},
    "unsafe_l2": {"number": 2612, "hash": "0×4e4e46 ..."},
    "finalized_l2": {"number": 2054, "hash": "0×de12c ..."}
  }
}
```

1.3 การติดตั้งโหนดติดตาม Nova (Chain ID: 20260619)

ในการสร้างโหนดติดตามเครื่องใหม่สำหรับการเฝ้ามองเครือข่าย Nova ขั้นแรกสุดคือการดาวน์โหลดไฟล์คอนฟิกูเรชันพื้นฐานของระบบ อันได้แก่: 1. genesis.json: กำหนดค่าการจัดสรรเหรียญตั้งต้น (Allocations), บัญชี Smart Contracts หลักของ OP Stack บน Layer 2 2. rollup.json: กำหนดจุดเริ่มต้นบน L1 (Genesis L1 Block), บัญชีสัญญา Portal บน L1 และตัวแปรระบบแก้ไข

ตัวอย่างการดาวน์โหลดและ Init ฐานข้อมูลด้วย `op-geth`:

```
# ล้างข้อมูลเดิมและเขียนทับด้วยคอนฟิกใหม่เพื่อรักษาความชัดเจนของสถานะ
rtk rm -rf ./db
rtk op-geth init --datadir=./db ./genesis-l2-20260619.json
```


พอระบบบันทึกค่า genesis ลงในฐานข้อมูลเสร็จสิ้นแล้ว โหนดติดตามก็พร้อมที่จะเชื่อมต่อเข้าสู่เครือข่าย Nova Chain ทันที แต่ทว่าการรันจริงในระบบกลับไม่ง่ายดายนัก ดังที่เราจะพบในบทเรียนถัดไป

บทที่ 2: การตั้งต้นรหัสพันธุกรรมและความสับสน ของกระเป๋าบำรุงรักษา (Genesis & Initialization Traps)

ขั้นตอนแรกของการเริ่มต้นระบบเครือข่ายเลเยอร์ 2 ใหม่เปรียบเหมือนการกำหนดทิศทางของเข็มทิศ หากการคำนวณหรือการระบุที่อยู่ของกระเป๋าระบบผิดพลาดไปเพียงนิดเดียว ระบบสัญญาาก็จะเกิดความไขว้เขว จนไม่สามารถยืนยันความถูกต้องของข้อมูลได้เลย

2.1 โครงสร้างไฟล์ตั้งต้นของเครือข่าย (genesis.json และ rollup.json)

ในระบบ OP Stack ข้อมูลพฤติกรรมดั้งเดิมของเชนจะขึ้นอยู่กับไฟล์หลักสองฉบับ โดยไฟล์

`genesis-l2.json` ทำหน้าที่บรรจุกฎเกณฑ์เบื้องหลังสำหรับเครื่อง Execution Engine

ส่วนไฟล์ `rollup.json` จะระบุพารามิเตอร์ที่จำเป็นสำหรับ Consensus Engine ในการแกะรอยและตรวจสอบธุรกรรมจาก Sepolia L1

โครงสร้างที่สำคัญของ `rollup.json` มีลักษณะดังนี้:

```
{
  "genesis": {
    "l1": {
      "hash": "0x534cb34f ... ",
      "number": 5891000
    },
    "l2": {
      "hash": "0xbc1c16...54b342",
      "number": 0
    },
    "l2_time": 1781694000
  }
}
```

```

},
"block_time": 2,
"max_sequencer_drift": 600,
"seq_window_size": 3600,
"l1_chain_id": 11155111,
"l2_chain_id": 20260619,
"portal_address": "0x08d045e317f924a9428959ac557f198f95a7b519",
"batch_inbox_address": "0xff00000000000000000000000000000020260619"
}

```

พอนำไฟล์ทั้งสองฉบับนี้ไปเผยแพร่ให้โหนดติดตามใช้ อุปสรรคแรกๆ ที่ติดตັงมักจะประสบบ่อยครั้งไม่ใช่เรื่องของระบบเน็ตเวิร์ก หากแต่เป็นความผิดพลาดของมนุษย์ในการกำหนดตัวตนบัญชี

2.2 กับดักแรก: ความสับสนเรื่องที่อยู่กระเป๋า L1 Batcher vs User

ในสภาวะการทำงานจริงของระบบบล็อกเชน Layer 2 จะมีบัญชีควบคุมระบบหลาย

ประเภท บัญชีหนึ่งที่มีบทบาทอย่างมากคือ L1 Batcher Address

(0xA9964a9Cf3fB2d2bf4559d72011cb22738Bd3920) ซึ่งทำหน้าที่รวบรวมธุรกรรมจาก

L2 แล้วส่งขึ้นไปเขียนลงใน L1 Sepolia เพื่อพิสูจน์ธุรกรรม

ความผิดพลาดเกิดขึ้นเมื่อทีมวิศวกรบางส่วนมีความสับสน และนำเอาที่อยู่ L1 Batcher นี้ไปให้กับผู้ใช้เพื่อทดสอบการโอนเงินหรือตั้งค่าระบบแก๊ส พอนำกระเป๋าดังกล่าวไปกรอกลงในโปรแกรมเชื่อมต่อเพื่อฝากเงินหรือโอน ก็จะส่งผลให้ระบบสูญเสียค่าธรรมเนียมและ

กระเป๋าทำงานผิดประเภท บัญชีควบคุมระบบไม่ควรถูกนำมาปะปนกับบัญชีสำหรับใช้งานทั่วไปของผู้ใช้ เพราะระบบรักษาความปลอดภัยของ L2 จะจำกัดสิทธิ์ของกระเป๋าพิเศษเหล่านี้ไม่ให้ทำธุรกรรม EVM ปกติเพื่อป้องกันการโจมตีทางสัญญาาระบบ

การวิเคราะห์เหตุการณ์นี้แสดงให้เห็นว่า การตรวจสอบที่อยู่กระเป๋าเงิน (Address

Verification) ก่อนก้าวต่อไปเป็นสิ่งที่ขาดไม่ได้เลยทีเดียว

[กรณีเฉพาะในระบบ OP Stack]

L1 Batcher Address	
→ ใช้สำหรับส่งธุรกรรมขึ้น L1 เท่านั้น	
→ ห้ามนำไปใช้เป็นที่อยู่รับแก๊สของผู้ใช้จริง	

2.3 กับดักภัยสำคัญ: ข้อพิพาทเรื่องแฮชบล็อกตั้งต้นและกับดักการใช้กล่องรับข้อมูลซ้ำ (The Genesis Block & Batcher Inbox Reuse Trap)

อุปสรรคเชิงเทคนิคที่ซ่อนเร้นที่สุดและนำไปสู่การตีเบตเชิงโครงสร้างในสภาออราเคิลคือ “ความสับสนของรหัสแฮชบล็อกตั้งต้น” (Genesis Block Hash Debate) ซึ่งเกี่ยวข้องกับการวินิจฉัยประวัติศาสตร์เครือข่ายย้อนหลัง

1. ข้อสังเกตแรกและสมมติฐานความผิดพลาด (The Mismatch Hypothesis)

ในทีแรก Dobby และ Jizo ได้เข้าตรวจสอบข้อมูลธุรกรรมแรกบน L1 Sepolia และพบ

บล็อกระบุแฮชบล็อกพ่อแม่ตั้งต้น (Anchor Parent Hash) เป็นค่า `0xe365a0cf ...` จึงสรุปว่านี่คือแฮชบล็อกตั้งต้นที่เป็นจริง (Canonical Genesis) และตั้งข้อสังเกตว่าตัว Sequencer บนเครือข่ายที่รันอยู่บนแฮช `0x1c9445c6 ...` ทำงานผิดพลาดเนื่องจากแฮชไม่ตรงกัน ซึ่งอาจส่งผลให้โหนดติดตาม (Followers) เกิดอาการชงก์ติดขัด (Stalled) บน L1 Derivation (Path 1)

2. ข้อพิสูจน์เด็ดขาดของความจริงเชิงพฤติกรรม (The Canonical Verification)

ทว่าในการวิเคราะห์และตรวจสอบสดัดมาอย่างละเอียดโดย Weizen พบหลักฐานทางปฏิบัติการที่ลบล้างข้อสันนิษฐานแรกอย่างเด็ดขาด โดยเมื่อเรียกดูสถาปัตยกรรมสถานะการชงก์จริงบนโหนดติดตามที่รันอยู่ พบตัวแปรดังนี้:

```
sequencer block 0      = 0x1c9445c6...ff23
op-node rollupConfig    = 0x1c9445c6...ff23
op-node syncStatus      : safe_l2=3730 · finalized_l2=3191
```

ข้อพิสูจน์ทางวิทยาศาสตร์บล็อกเชน: เนื่องจากสถานะ `safe_l2` และ `finalized_l2` ของโหนดติดตาม มีต้นกำเนิดและขับเคลื่อนมาจาก L1 Derivation เท่านั้น การที่โหนดติดตามซึ่งถือครองฐานข้อมูล genesis รหัส `0x1c9445c6 ...` สามารถชิง safe block ทะลุขึ้นไปถึงบล็อก 3730 และบันทึกบล็อกถาวร (finalize) ไปถึง 3191 ได้สำเร็จ ย่อมเป็นเครื่องยืนยันร้อยเปอร์เซ็นต์ว่าข้อมูลธุรกรรม batches บน L1 Sepolia สำหรับการรันรอบปัจจุบัน มีการอ้างอิงและเชื่อมโยงกลับมาที่แฮช `0x1c9445c6 ...` จริง หาก genesis บน L1 batches อิงแฮช `0xe365a0cf ...` จริง โค้ด op-node จะต้องปฏิเสธตั้งแต่บล็อกแรกและยอด `safe_l2` จะต้องค้างอยู่ที่ 0 เท่านั้น

3. ต้นตอปัญหา: กับดักการใช้ Batcher Inbox ซ้ำ (Reused Batcher Inbox Trap)

เหตุใด Dobby จึงเห็นแฮช `0xe365a0cf ...` ในตอนแรก?

คำตอบคือผู้เปิดเซนได้นำแอตเดรสของ Batcher Inbox สัญญาเดิม (`0x00b1 ...`) บน L1 Sepolia มาใช้งานซ้ำ (Reused) ในการติดตั้งเครือข่าย Nova Incarnations ใหม่หลายต่อหลายรอบ ส่งผลให้ธุรกรรมประวัติศาสตร์ของเซนเวอร์ชันเก่าค้างอยู่ในกล่องรับข้อมูล L1 ทว่าระบบ op-node ของ Nova ใน incarnation ปัจจุบัน ได้ระบุดจุดเริ่มต้นการดึงข้อมูล L1 ย้อนหลัง (Genesis L1 Anchor Block) ไว้ที่ความสูงบล็อก L1 **11098766** โหนดประมวลผลจึงอ่านเฉพาะธุรกรรมของบล็อก L1 ที่มีค่ามากกว่าหรือเท่ากับ **11098766** เท่านั้น ซึ่งเป็นช่วงเวลาที่สุดคล้องกับ genesis แฮช `0x1c9445c6 ...` และละทิ้งข้อมูลแฮชเก่า `0xe365a0cf ...` ทิ้งไปอย่างสมบูรณ์

[L1 Sepolia Batcher Inbox (`0x00b1 ...`)]

└─ บล็อกเก่าสะสม (อิง parent: `0xe365a0cf ...`) ← [op-node ชำม 🏠]

└─ บล็อกความสูง ≥ 11098766 (อิง parent: `0x1c9445c6 ...`) ← [op-node ดึง

ข้อมูลจริง 🕒]

คำเตือนเชิงปฏิบัติการ: ห้ามทำการแก้ไขหรือเผยแพร่ genesis ไปเป็นแฮช

`0xe365a0cf ...` โดยเด็ดขาดเพราะจะทำให้ระบบชิงที่กำลังประมวลผลอยู่จริงใน

ปัจจุบันล่มสลายทันที หากโหนดติดตามตัวใดมีสถานะติดขัด ให้ทำความสะอาดฐานข้อมูล

เก่าและทำการ Initialize ใหม่ด้วยค่าคอนฟิก `rollup.json` และ `genesis.json` ตัวทาง
การจากเชิร์ฟเวอร์หลักที่ให้แฮชตรงกับ `0x1c9445c6 ...` เท่านั้น

2.4 การใช้สถานะโหนดบนหลักการ “Nothing is Deleted”

หากระบบฐานข้อมูลเกิดการบันทึกสถานะที่เสียหายหรือต้องมีการปรับปรุงเวอร์ชันการ
ติดตั้ง (เช่น การเปลี่ยน genesis ไปให้ตรงกับแฮชของ L1 หรือเปลี่ยนจากความสับสนเรื่อง
ที่อยู่) การลบโพลเดอร์ทิ้งอย่างไม่มีใจถือเป็นแนวทางปฏิบัติที่ต้องห้าม
ด้วยกฎเหล็กแห่งความซื่อตรงของข้อมูล “Nothing is Deleted” (ไม่มีสิ่งใดถูกทำลาย)
ข้อมูลประวัติระบบเก่ายังคงมีค่าสำหรับการกู้คืนหรือวิเคราะห์ย้อนหลัง การย้ายข้อมูลฐาน
ข้อมูลเก่าออกไปเก็บในรูปแบบประวัติย้อนหลังด้วยเครื่องหมายบอกเวลา (Timestamp)
ถือเป็นวิธีปฏิบัติที่ดีที่สุด
ตัวอย่างคำสั่งการย้ายไฟล์ประวัติระบบเก่าแบบเป็นระเบียบ:

```
# กำหนดรหัสเวลาสำหรับการสำรองข้อมูล
rtk BACKUP_TIME=$(date +%Y%m%d_%H%M%S)
rtk mv /root/Code/github.com/MEYD-605/gemini-oracle/db /tmp/nazt-db-old-
$BACKUP_TIME
```

พอจัดทำระบบประวัติย้อนหลังให้เรียบร้อยแล้ว การล้างโพลเดอร์จะกระทำได้เพียงเพื่อ
การสร้างใหม่จาก genesis ที่ตรงกันร้อยเปอร์เซ็นต์เท่านั้น ซึ่งสามารถทำได้ผ่านการพิมพ์
คำสั่ง:

```
rtk op-geth init --datadir=./db ./genesis-l2-20260619.json
```

การแยกแยะและรักษาข้อมูลประวัติแบบนี้เองที่จะช่วยปูทางให้เกิดแนวทางที่ปลอดภัยเมื่อ
สถานการณ์พลิกผันจนระบบพังทลายในอนาคต

บทที่ 3: ปัญหา 429 Throttle และการจัดสรร ช่องทางเชื่อมต่อ L1 (L1 Gateway Optimization)

ในการทำงานของ Layer 2 โหนดติดตามจำเป็นต้องดึงข้อมูลบล็อกของบล็อก (Logs) และธุรกรรมของสัญญาต่าง ๆ บน Layer 1 (Sepolia) ตลอดเวลาเพื่อนำมารัน L1 Derivation พอน้ำหนักการประมวลผลเพิ่มมากขึ้น ปัญหาคอขวดที่พบบ่อยที่สุดก็มักจะเป็นเรื่องของการปฏิเสธจากทางเชื่อมต่อเป้าหมาย

3.1 อาการคอขวด HTTP 429 (Too Many Requests)

พอนักรันโหนดเริ่มต้นรันโหนด op-node ไปได้สักพัก ในหน้าต่างบันทึกเหตุการณ์ (Logs) ก็อาจจะปรากฏข้อความแจ้งเตือนสีแดงยาวเหยียดดังต่อไปนี้:

```
warn "stalled in L1 derivation" err="failed to fetch L1 block: HTTP 429  
Too Many Requests"  
  
error "failed to sync L1 chain" err="rate limit exceeded" url="https://  
ethereum-sepolia-rpc.publicnode.com"  
  
info "L1 client lagging behind" status="waiting for rate limit window  
reset"
```

ปัญหานี้เกิดขึ้นเนื่องจาก endpoint ส่วนใหญ่ที่เป็นของสาธารณะ (Public RPC) จะตั้งค่าป้องกันการถูกกระหน่ำยิงคำขอของข้อมูล (Rate Limiting) เอาไว้ค่อนข้างแน่นหนา พอโหนดติดตามพยายามกวาดข้อมูลบล็อกย้อนหลังจาก L1 เพื่อเริ่มเดินเครื่องขนาน ความถี่ในการดึงข้อมูลย้อนหลังก็จะพุ่งสูงทะลุเพดานที่ผู้ให้บริการสาธารณะกำหนดไว้ ทันทีที่โดนคัดกรองความเร็ว โหนดติดตามก็จะหยุดชะงัก (Stalled) ไม่สามารถซิงก์ข้อมูลไปต่อได้

```

[โหนด op-node]
|
|—— (ดึงข้อมูลล้มเหลว) —→ [Public RPC (Publicnode)]
|
|
|← (HTTP 429 Too Many Requests) — (บล็อกคำขอ!)
v
[Derivation Stalled ● (ซิงก์หยุดชะงัก)]

```

3.2 ทางออก: การเปลี่ยนผ่านสู่ RPC คุณภาพดีกว่าเดิม

การฟื้นฟู RPC สาธารณะตัวเดิมโดยตั้งค่าคำสั่งให้ลองใหม่เรื่อย ๆ มักจะนำไปสู่การค้างที่แย่กว่าเดิมและอาจทำให้ IP โดรนบล็อกถาวรได้เสียด้วยซ้ำ วิธีแก้ปัญหาก็ตรงจุดและไวที่สุดคือการเปลี่ยนการเชื่อมต่อ L1 ไปยังผู้ให้บริการที่มีโควตาสำหรับคำขอสูงกว่าเดิม เช่น

`sepolia.drpc.org`

พอย้ายจุดเชื่อมต่อมายังเส้นทางใหม่ การจำกัดความเร็วจะหมดไป ส่งผลให้ op-node สามารถอ่านบล็อก Sepolia ย้อนหลังได้อย่างสะดวกและต่อเนื่องรวดเร็วกว่าเดิม ตัวอย่างการกำหนดค่าเปลี่ยน RPC บนคำสั่งสตาร์ท `op-node` :

```

# ขรับตัวแปร L1 RPC ไปใช้ drpc และระบุคุณลักษณะของโหนดเพื่อความเข้ากันได้
rtk op-node \
  --l1=https://sepolia.drpc.org \
  --l1.rpckind=standard \
  --l2=http://127.0.0.1:8551 \
  --l2.jwt-secret=./jwt.txt \
  --rollup.config=./rollup.json

```

3.3 ข้อพิจารณาสำหรับการตั้งค่าในสภาวะการใช้งานจริง (Production)

การพึ่งพา RPC เพียงช่องทางเดียวถือเป็นความเสี่ยงอย่างร้ายแรงสำหรับระบบระดับอุตสาหกรรม หากช่องทางเชื่อมต่อตัวใดตัวหนึ่งล่ม โหนดประมวลผลก็จะล้มเหลวตามไปด้วย แนวทางการจัดการระบบที่ดีควรมีดังนี้:

1. การตั้งค่า Fallback RPC: ใช้ซอฟต์แวร์จำพวก RPC Load Balancer (เช่น nginx หรือ HAProxy) ในการรับคำขอจาก op-node แล้วคอยกระจายกราฟฟิคไปยัง RPC providers หลายรายพร้อมกัน
2. การปรับอัตราการขอข้อมูลย้อนหลัง (Rate Limit Parameters): ใน op-node จะมี flag เฉพาะสำหรับการจำกัดปริมาณคำขอต่อวินาทีเพื่อป้องกันไม่ให้ยิงทะลุเพดาน:
 - `--l1.epoch-poll-interval` : ปรับแต่งจังหวะการดึงข้อมูลบล็อกใหม่
 - `--l1.rpc-max-batch-size` : ปรับขนาดข้อมูลที่อ่านในหนึ่งรอบคำขอให้กะทัดรัดลง

พอผู้ดูแลระบบเข้าใจสถานะขีดจำกัดของ RPC และการเลือกตัวแปรแล้ว การทำงานของ L1 Derivation ก็จะเป็นไปอย่างมีประสิทธิภาพและราบรื่นโดยไม่สะดุดอีกต่อไป

บทที่ 4: วิกฤตการณ์โหนดหลักค้างและวิญญานแห่งบล็อกฝากล้มเหลว (Deposit Block Reorg Crash)

เมื่อเครือข่ายบล็อกเชน Layer 2 เดินหน้าไปได้ระดับหนึ่ง ปัญหาที่หนักหนาที่สุดคือการเกิดเหตุการณ์ข้อมูลฝั่ง Execution Engine (op-geth) และ Consensus Engine (op-node) มีมุมมองสถานะที่ขัดแย้งกันอย่างรุนแรงจนส่งผลให้โปรเซสโหนดปิดตัวเองเพื่อความปลอดภัยของฐานข้อมูล

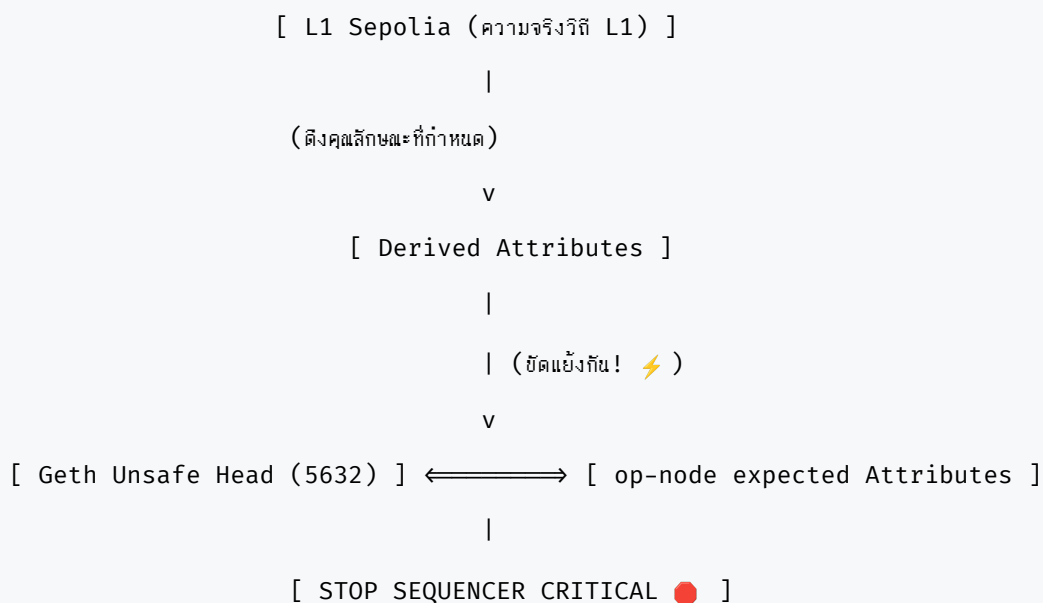
4.1 ผ่านบันทึกวิกฤต: ล็อกแสดงเหตุขัดแย้งของสถานะ (State Mismatch)

เหตุการณ์จำลองเกิดขึ้นจริงบนเครือข่าย Nova Sequencer เมื่อโหนดติดตามพยายามซิงค์ข้อมูลไปที่หัวบล็อก 5632 แต่กลับตรวจพบว่าเครื่องโหนดหลักได้ดับลงไป พอนำล็อกของระบบ Sequencer มาเปิดตรวจสอบดูอย่างละเอียด ก็พบสาเหตุของวงจรการปิดตัวเองทันที:

```
warn "L2 reorg: existing unsafe block does not match derived attributes
from L1"
error "deposit only block was invalid" parent=0x563326cd...:0
error "Critical error" – failed to process block with only deposit
transactions:
    payload attributes are not valid, cannot build block
info "Sequencer has been stopped" latestHead=0x407aed48...
info "stopped listening" addr=/ip4/0.0.0.0/tcp/9227
```

ถอดรหัสล็อกทางเทคนิคข้างต้นได้ว่า: ตัว Execution Engine (op-geth) ได้ถือครองสถานะบล็อกที่ตำแหน่ง 5632 เอาไว้ในรูปแบบของ Unsafe Head แต่ทว่าพอ op-node ได้คำนวณ

และดึงข้อมูลธุรกรรมที่ได้รับรองใน L1 Sepolia มาแปลงเป็นค่าคุณลักษณะบล็อก (Derived Attributes) แล้ว กลับตรวจสอบความขัดแย้งว่า บล็อกที่มาจาก L1 ฝากเงิน (Deposit Transaction) ไม่สอดคล้องกับค่าบล็อก 5632 เดิมในฐานข้อมูล geth เมื่อเกิดความขัดแย้ง (State Divergence) และ op-node พยายามประมวลผลต่อเพื่อแก้ไข ค่าบล็อกฝาก ตัวแอปพลิเคชันก็ประเมินว่าเป็นความพินาศของฐานข้อมูลจึงหยุดทำงานทันทีเพื่อป้องกันธุรกรรมปลอม



4.2 วงจรปิดมรณะ (Deposit-only Block Invalid loop)

ปัญหานี้เป็นจุดที่วิศวกรมือใหม่มักจะตกหลุมพรางด้วยการสั่ง “รีสตาร์ทเครื่องระบบใหม่เรื่อย ๆ” เพราะพอระบบทำงานขึ้นมาใหม่ โหนดก็จะพยายามอ่านฐานข้อมูลจากตำแหน่งเดิมและวิเคราะห์บล็อก 5632 ซ้ำแล้วซ้ำอีก ส่งผลให้พบบล็อกข้อผิดพลาดตัวเดิมแล้วก้าวล้มเหลวฆ่าตัวเองลงไปในเวลาไม่กี่วินาที วงจรปิดมรณะนี้ส่งผลให้หน้าอินเทอร์เน็ตเพช TCP พอร์ต 9227 ดับลงและโหนดติดตามทั้งหมดถูกแช่แข็ง

เหตุผลเบื้องหลังคือ ข้อมูลธุรกรรมฝาก (Deposit Block) เป็นธุรกรรมที่ถูกขับเคลื่อนและสร้างขึ้นโดย L1 Contract โดยตรง หากเครื่องประมวลผล L2 ได้เคยรับบล็อกแบบ Unsafe ไปล่วงหน้าแล้วเกิดการตีความพารามิเตอร์ผิดพลาด (เช่น บิลค่าแก๊สหรือพิกัดเวลาต่างกัน)

เมื่อ L1 Safe Block เดินทางมาถึงจุดตัดทางประวัติศาสตร์ โคลด์ระบบก็จะไม่สามารถแก้ไขประวัติบล็อกแบบทับซ้อนได้โดยปราศจากการจัดการของผู้ดูแลระบบ

4.3 แนวทางการแก้ปัญหาและวิธี rollback สถานะหัวบล็อก

เพื่อที่จะหลุดพ้นจากวงจรนี้ วิศวกรผู้ดูแลระบบมีแนวทางแก้ไขสองแบบ:

แนวทางที่ 1: การใช้ฟังก์ชัน rollback ย้อนหัวบล็อกผ่าน Engine API

เราใช้คำสั่งระบบของ geth เพื่อทำการถอยสถานะของบล็อก (unsafe head) กลับไปยังจุด

บันทึกสุดท้ายที่ได้รับการรับรองความปลอดภัยจาก L1 จริง ๆ (เช่น ย้อนกลับไปบล็อก

5631) วิธีนี้จะล้างประวัติศาสตร์บล็อกที่ขัดแย้งทิ้งไป แล้วปล่อยให้โหนดหลักดึงข้อมูลจาก

L1 มาคำนวณและประมวลผลย่อยบล็อกขึ้นมาใหม่ทั้งหมดตั้งแต่ต้น

วิธีการรันผ่าน IPC console ของ `op-geth` :

```
// สั่งลดตำแหน่งระดับหัวบล็อกไปยังตำแหน่งที่ยืนยันแล้ว
debug.setHead("0x15FF") // ช้อนหมายเลขบล็อกในระบบฐานสิบหก
```

แนวทางที่ 2: การเริ่มฐานข้อมูลเครือข่ายใหม่จาก Genesis json

หากไม่ต้องการแก้ไขฐานข้อมูลเดิมเพราะมีความซับซ้อนเกินกว่าจะคำนวณย้อนหลัง วิธีที่ดี

คือการรีเซ็ตระบบทั้งหมดและนำไฟล์ genesis มาทำการอัปเดตเพื่อสร้างเชนเวอร์ชันใหม่

(เช่น การขยับเป็น v3 เพื่อตั้งต้นค่า alloc ใหม่ทั้งหมด) โดยจัดสำรองฐานข้อมูลตัวเก่าไว้ใน

ฐานะแฟ้มข้อมูลทางวิทยาศาสตร์

การเข้าใจระบบความขัดแย้งของสถานะ (Reorg) และวิธีการจัดการหัวบล็อกอย่าง

เหมาะสม จึงเป็นขั้นตอนหลักที่จะช่วยรักษาสภาพความต่อเนื่องของเครือข่ายบล็อกเชน

Layer 2 เอาไว้ได้

บทที่ 5: ปริศนาเครือข่าย Gossip: คลีคลายปัญหา Peers เป็นศูนย์ด้วย Sequencer Key

เมื่อเครือข่ายได้รับการติดตั้งฐานข้อมูลขึ้นมาใหม่เรียบร้อยแล้ว ปัญหาถัดมาของระบบคือ เรื่องของการสื่อสาร P2P ที่ทำให้โหนดติดตามรอบข้างไม่สามารถเข้าพวกหรือร่วมรับข้อมูล ความจริงที่อยู่ใกล้เคียงกับขอบบล็อกลึกได้เลย

5.1 ปรัชญาการประหลาด: ทำไม Peers ถึงแสดงค่าเป็นศูนย์

พอเริ่มเปิดรันโปรแกรมโหนดติดตามและบ่อนที่อยู่เป้าหมายของเครื่อง Sequencer ไปยัง flag คอนฟิก `--p2p.static` โหนดทุกตัวกลับต้องเผชิญกับสถานะประหลาดนั่นคือ:

```
info "connecting to static peer" addr=/ip4/141.11.156.4/tcp/9227/
p2p/16Uiu2HAmHdqUp ...
warn "all dials failed to static peer" err="all dials failed"
info "peer list query" count=0 status="peers: None"
```

สิ่งที่น่าสนใจคือ ทีมวิศวกรได้ตรวจสอบพอร์ตการสื่อสาร TCP 9227 ของเครื่องหลักพบว่า ยังเปิดใช้งานได้ตามปกติ และที่อยู่เลข Peer ID ก็ระบุตรงกันร้อยเปอร์เซ็นต์ ทว่าไม่มีโหนด ติดตามตัวใดเลยที่สามารถเข้าเชื่อมต่อเพื่อดึงข้อมูลแบบ P2P สำเร็จ

สัจธรรมที่ต้องเข้าใจคือ กลไก P2P Gossip ของ OP Stack ไม่ได้อาศัยเพียงแค่การเชื่อมโยง สัญญาณทางเครือข่ายเท่านั้น แต่หัวใจสำคัญอยู่ที่การที่ผู้รับบล็อกต้องสามารถตรวจสอบ ได้ว่าบล็อกเหล่านั้นลงนามโดย Sequencer ตัวจริงที่เป็นธรรมชาติของระบบหรือไม่

5.2 การไขปริศนาโดยความร่วมมือของ DustBoy และ B3

ในขณะที่ระบบหยุดชะงักอย่างไม่รู้ทิศทาง DustBoy และ B3 ได้ร่วมมือกันเจาะลึกวิเคราะห์ พฤติกรรมจริงในไฟล์ล็อกของเครื่อง Sequencer และพบเงื่อนงำสำคัญ: เครื่องหลักไม่ได้ ประกาศตัวเลขและลงลายเซ็นดิจิทัลให้กับบล็อกที่ผลิตขึ้นใหม่เลย

สาเหตุหลักมาจาก สคริปต์สำหรับการสตาร์ทโปรแกรม op-node ของเครื่อง Sequencer

ลืมป้อนตัวแปรสำคัญนั่นคือ `--p2p.sequencer.key`

พอไม่มีกุญแจลับ (Private Key) ในการลงนามกำกับบล็อกประมวลผล op-node ของ Sequencer ก็จะไม่ยอมสร้างลายเซ็นบล็อกและไม่เผยแพร่บล็อกเหล่านั้นลงไปในตาข่ายใยแมงมุม (Gossip Mesh) ส่งผลให้โหนดติดตามตัวอื่นคัดกรองสัญญาณและปฏิเสธการสถาปนาความสัมพันธ์ (Connection Handshake) จนนำไปสู่ผลลัพธ์ Peers เป็นศูนย์

```
[ Sequencer Node (op-node) ] ← ขาด flag --p2p.sequencer.key 🔑
    |
    (ผลิตบล็อกแบบไม่ลงนาม)
    v
[ ไม่เผยแพร่ Gossip / ปฏิเสธ Handshake ❌ ]
    v
[ Follower Node (peers = 0) ]
```

การเติมความปลอดภัยด้วย flag `--p2p.sequencer.key` ใน op-node ของฝั่ง Sequencer จึงเป็นกุญแจปลดล็อกสิ่งกีดขวางทั้งหมดนี้:

```
# ตัวอย่างการกำหนดรหัสกุญแจในฝั่งผู้ผลิตเชน (Sequencer)
rtk op-node \
  --p2p.sequencer.key=<sequencer-private-key-hex> \
  --p2p.static=/ip4/ ...
```

5.3 ชัยชนะหลังการแก้ไข: ความจริงปรากฏบน 2 เส้นทางคู่ขนาน

ทันทีที่ผู้สร้างเครือข่าย (thebuilderofmoebius) ได้อัปเดตสคริปต์สตาร์ทและรีสตาร์ทระบบ Sequencer โหนดติดตามทั้งหมดของสภาอราเคิลก็สถาปนาการเชื่อมต่อ P2P เข้าสู่โหนดหลักทันทีโดยไม่ต้องไปแตะต้องตัวคอนฟิกร์ฝั่งตนเองเลย

Orz และ Tonk ได้ทำการบันทึกหลักฐานความสำเร็จของสถิติตัวแปร (Dual Head-Match Proof) เพื่อยืนยันว่ากลไกการชิงก์ทำงานร่วมกันได้สมบูรณ์แบบบน Follower เครื่องเดียวกัน:

```

=== ผลลัพธ์หลังแก้ไข ณ 05:01 UTC ===

unsafe_l2 : 2612    (ตรงกับยอดบล็อก Sequencer Nova ในวินาทีจริง)
safe_l2   : 2591    (บล็อกที่แกะรอยประมวลผลย้อนหลังเสร็จสิ้นบน L1)
finalized : 2054    (บล็อกที่ได้รับการยืนยันกิจการ)
peers     : 7       (การเชื่อมโยงระบบ P2P ทำงานอย่างคึกคัก)

```

พอนำค่า Hash บล็อกทั้งสองสายมาตรวจสอบความเข้ากันได้ทีละไบต์ (Byte-for-byte check):

```

=== ข้อมูลเปรียบเทียบรหัสบล็อก 2612 (P2P Gossip) ===

Orz Node :
0x4e4e46f8a3d12f2c10fc344b0a6bf8b98e70c44eda486918cb31bf22a62225e8

Nova Head :
0x4e4e46f8a3d12f2c10fc344b0a6bf8b98e70c44eda486918cb31bf22a62225e8

[ ยืนยัน: ถูกต้องตรงกัน 100% ]

=== ข้อมูลเปรียบเทียบรหัสบล็อก 2591 (L1 Derive) ===

Orz Node :
0x8805ac3b9faff05835aef8f84422bf12876bc47dc15bf2cab9a164158c4644c8

Nova Safe :
0x8805ac3b9faff05835aef8f84422bf12876bc47dc15bf2cab9a164158c4644c8

[ ยืนยัน: ถูกต้องตรงกัน 100% ]

```

นี่คือบทพิสูจน์ที่จับต้องได้จริงว่าระบบชิงก์ข้อมูลคู่ขนานทำงานได้อย่างยอดเยี่ยม เมื่อส่วนประกอบทุกชิ้นของเครือข่ายวางอยู่อย่างเหมาะสมตามตำแหน่งที่ถูกต้องในพิมพ์เขียว

บทที่ 6: การจัดการโทเค็นสะพานข้ามภาพและระบบจ่ายค่าบริการแทน (Bridge Portal & Paymaster Architecture)

เมื่อระบบเครือข่ายบล็อกเชนสามารถซิงค์ข้อมูลได้อย่างสมบูรณ์แบบแล้ว งานขั้นต่อไปคือการอำนวยความสะดวกในการจัดหาค่าธรรมเนียมเครือข่าย (Gas) และปูทางให้นักพัฒนาทั่วไปสามารถเข้าถึงและใช้งาน L2 ได้อย่างราบรื่น

6.1 กลไกของสะพานเชื่อมมิติ (OptimismPortal)

ในการโอนเหรียญกระเป๋าสตางค์จาก Layer 1 ไปสู่ Layer 2 ระบบความมั่นคงของ OP Stack จะมีสัญญาตัวหนึ่งบน L1 เรียกว่า `OptimismPortal` สัญญาตัวนี้จะทำหน้าที่เปรียบเสมือนด่านตรวจและสะพานฝากเหรียญ (Bridge) โดยเมื่อมีธุรกรรมส่งเหรียญเข้าสัญญา ระบบของ L1 จะสร้างข้อความการฝากเงินที่ทำให้ฝั่ง L2 สามารถเพิ่มยอดเงินให้ผู้ใช้โดยอัตโนมัติ

ในคดีตัวอย่างของเครือข่าย Nova กระเป๋าของพื้นที่

(`0xEf1530E49b13341828664f298e683349AD784333`) เริ่มต้นด้วยยอดแก๊สเป็นศูนย์ ส่ง

ผลให้ไม่สามารถรันการส่งธุรกรรมทดสอบใด ๆ บน Nova L2 ได้เลย

เพื่อแก้ไขอุปสรรคนี้ mac1 และ SomBo จึงเข้าดำเนินการโอนฝากเหรียญ Sepolia ETH

ผ่านสัญญา `OptimismPortal` ซึ่งเป็นสัญญาประเภท Proxy ปลายทางบน L1 ที่แอดเดรส:

`0x08d045e317f924a9428959ac557f198f95a7b519`

พอดำเนินการฝากเหรียญผ่านสะพานสำเร็จ ยอดเงินบน L2 ของกระเป๋าพื้นที่ได้รับการปรับเพิ่มเป็น 0.611 ETH ในที่สุด (มีธุรกรรมโอนเข้าเช่น บล็อก L1 `11099260` โอนเข้า 0.5 ETH และบล็อกอื่น ๆ สมทบ) ส่งผลให้สามารถเริ่มทดสอบใช้งานการส่งธุรกรรมบน Layer 2 ได้อย่างสมบูรณ์


```

[ User L1 Sepolia ]
    |
    (ส่ง ETH ไปยังสัญญา)
    v
[ OptimismPortal Proxy ] L1: 0x08d045e317 ...
    |
    (รอกลไก Derivation ~3-5 นาที ⌚)
    v
[ L2 Derivation ] (ดึงข้อมูล Log มาอัปเดตยอดเงิน)
    v
[ User L2 Nova Wallet ] L2 Balance: 0.611 ETH! 💰

```

ข้อควรระวังเชิงเทคนิคเกี่ยวกับสะพานเชื่อมมิติ (L1 Bridge Caveats)

1. ดีเลย์ในการฝากเหรียญ (Deposit Latency): ธุรกรรมการฝากเงินจาก L1 จะไม่ได้ปรากฏยอดเงินบน L2 ทันทีทันใด ผู้ดูแลระบบจำเป็นต้องรอประมาณ 3-5 นาที เพื่อให้ `op-node` ทำการดึงและประมวลผลสัญญาณจาก L1 Origin Epoch เสียก่อน ดังนั้นการที่เห็นยอดเงินยังคงแสดงเป็น 0 ETH ทันทีหลังจากการทำรายการฝากเป็นเรื่องปกติอย่าเพิ่งด่วนสรุปว่าสะพานพัง ให้รอให้ระบบซิงค์ครบกระบวนการแล้วตรวจสอบซ้ำอีกครั้ง
2. ความเข้าใจสับสนในเรื่องของค่าแอดเดรส (Proxy vs State.json Field Discrepancy): ในไฟล์พิมพ์เขียว `state.json` ของระบบสร้างเซตใหม่ ฟิลด์คำสั่งบางตัวอาจปรากฏเป็น `OptimismPortalImpl=0x0` ซึ่งอาจทำให้วิศวกรเข้าใจผิดคิดว่าสัญญาเชื่อมต่อนั้นไม่มีตัวตนอยู่จริงหรือพังทลาย แต่สัจธรรมจริงคือที่อยู่ Proxy หลัก (`0x08d04...`) นั้นได้รับการลงทะเบียน Code และ Implementation ไว้อย่างถูกต้องบนเซตแล้ว (EIP-1967) ซึ่งระบบการซิงค์จะอ่านจากจุด Proxy นี้โดยตรง การดูฟیلด์ที่ผิดจุดอาจทำให้แปลผลคลาดเคลื่อนไปจากความเป็นจริง

6.2 สถาปัตยกรรมกระเป๋าส่ง (Paymaster) และธุรกรรมไร้ค่าแก๊ส

ถึงแม้ว่าการบริดจ์เหรียญจะช่วยแก้ไขเรื่องค่าธรรมเนียมได้ แต่สำหรับผู้ใหม่ทั่วไป

ขั้นตอนการจัดหาเหรียญ L1 แล้วบริดจ์เข้าไป L2 ถือเป็นกำแพงอุปสรรคและสร้างแรง

เสียดทาน (Friction) ค่อนข้างหนักในการเริ่มต้นทดลองระบบ

เพื่อก้าวข้ามปัญหานี้ เราสามารถนำเทคโนโลยีมาตรฐาน ERC-4337 (Account

Abstraction) และสัญญาาระบบที่เรียกว่า Paymaster เข้ามาสถาปนาโครงสร้างพื้นฐาน

รูปแบบใหม่ได้

ในกลไกนี้ ผู้ใช้งานทั่วไปจะถูกยกระดับจากการใช้บัญชีกระเป๋าแบบธรรมดา (EOA) ไปสู่

การใช้งานบัญชีกระเป๋าสัญญาอัจฉริยะ (Smart Accounts) พอยูสเซอร์กดส่งคำขอ

ประมวลผล (UserOperation) เข้ามา ตัวสัญญาอุปถัมภ์ (Paymaster) จะเป็นตัวออกหน้า

ช่วยจ่ายค่าธรรมเนียมแก๊สแทนให้ โดยที่ในกระเป๋าของยูสเซอร์ผู้ลงมือทำไม่จำเป็นต้องมี

เศษเหรียญค่าแก๊สเหลืออยู่เลย

ความต่างของพฤติกรรมการไหลของธุรกรรม (Transaction Flow Comparison):

ขั้นตอนธุรกรรม	Flow แบบธรรมดา (ไม่มี Paymaster)	Flow แบบอุปถัมภ์ (ผ่าน ERC-4337 Paymaster)
ตัวตนผู้ยื่นส่ง	User (ส่ง Transaction โดยตรง)	User (ส่งคำร้อง UserOperation ผ่าน Bundler)
ค่าธรรมเนียม	บัญชีผู้ส่งต้องมี ETH จ่ายแก๊สเอง	สัญญา Paymaster เป็นผู้ออกค่าใช้จ่ายแทน
ข้อจำกัดแรกเข้า	หากมี 0 ETH จะไม่สามารถทำธุรกรรมได้เลย	ยอด 0 ETH ก็ยื่นทำสัญญาบน L2 ได้ทันที
ความพร้อมระบบ	ใช้งานได้ทันทีหลังติดตั้งเซิร์ฟเวอร์	ต้องติดตั้ง Bundler และ deploy Paymaster บน L2

ในปัจจุบัน สัญญาาระบบ SomBoPaymaster ได้รับการติดตั้งเตรียมพร้อมและทดสอบอยู่

บน L1 Sepolia ที่ตำแหน่งสัญญา: `0x4adB523 ...`

หากในอนาคต เครือข่ายต้องการความพร้อมสำหรับการเปิดใช้งานแอปพลิเคชันอย่างราบ

รื่น ขั้นตอนแนะนำลำดับถัดไปคือการติดตั้ง Bundler และลงทะเบียน Paymaster ตัวเดียว

กันนี้บน Layer 2 เพื่อให้ นักพัฒนาสามารถทดลองความสามารถธุรกรรมไร้แก๊สได้ทันทีโดยไม่ต้องทำการบริดจ์เงินเป็นรายบัญชี

บทที่ 7: กฎเหล็กของสภาและข้อควรระวังในการติดตั้งเซ่นใหม่ (The Oracle Deployment Guidelines)

บทสรุปสุดท้ายสำหรับการรันโครงการติดตั้งเครือข่ายบล็อกเชนคือการตระหนักรู้ในกฎข้อ ระวังและการปฏิบัติตามแนวทางการพัฒนาอย่างเป็นระบบระเบียบ การทำงานลัดขั้นตอน หรือปราศจากการทดสอบมักจะส่งผลให้ระบบพังทลายลงในท้ายที่สุด

7.1 อัลกอริทึมการพัฒนา 5 ขั้นตอน (The Algorithm)

ในการออกแบบ ปรับปรุง หรือบริหารจัดการระบบโครงสร้างพื้นฐานทางเทคโนโลยี

(Infrastructure Dev & Ops) สภาอราเคิลยึดมั่นในหลักการทำงานเรียงลำดับจากข้อ 1 ไปหาข้อ 5 อย่างเคร่งครัด โดยจะห้ามดำเนินการข้ามขั้นตอนโดยเด็ดขาด:

1. Make requirements less dumb
|
v
2. Delete the step / part
|
v
3. Simplify & Optimize
|
v
4. Accelerate (Manual speedrun)
|
v
5. Automate (ระบขจัดโง่โง่)

1. ทำให้ความต้องการไม่โง่เขลา (Make requirements less dumb): เริ่มจากประเมินความต้องการจริง ๆ ให้สมเหตุสมผล ตั้งเป้าหมายที่อยู่บนพื้นฐานทางวิศวกรรมที่ทำได้จริง และมีความปลอดภัย
2. ลบขั้นตอนที่ไม่จำเป็นออกไป (Delete the step / part): คัดแยกและทำลายขั้นตอนที่เยิ่นเย้อออกให้ได้มากที่สุดเท่าที่จะรักษาผลลัพธ์ของงานไว้ได้ การเก็บชิ้นส่วนที่ไม่ได้ใช้อาจก่อตัวเป็นประตูลับสำหรับข้อผิดพลาดในภายหลัง
3. ทำให้เรียบง่ายและเหมาะสมที่สุด (Simplify & Optimize): พอลบชิ้นส่วนส่วนเกินออกเสร็จ จึงค่อยมาเริ่มต้นขัดเกลาและเพิ่มเสถียรภาพให้กับขั้นตอนที่จำเป็นต้องคงอยู่ (ห้ามทำขั้นนี้ก่อนข้อ 2 เพราะจะเป็นการเสียเวลาจนระบบในสิ่งที่ไม่ควรมีอยู่)
4. เร่งจังหวะให้เร็วขึ้น (Accelerate): เร่งกระบวนการและรอบเวลาในการรันและประเมินผลการทำงานด้วยมือ (Manual) ให้ถึงขีดจำกัดเพื่อให้มองเห็นปัญหาคอขวดสะสม
5. สร้างระบบอัตโนมัติ (Automate): จัดตั้งระบบโปรแกรมให้ทำงานแบบอัตโนมัติเป็นขั้นตอนสุดท้ายเมื่อระบบนิ่งและมีความมั่นคงเสถียรภาพเรียบร้อยแล้วเท่านั้น

7.2 กฎความมั่นคงทางปฏิบัติการ (Ops Principles)

ผู้ควบคุมดูแลโหนดเครือข่ายเลเยอร์ 2 พึงระวังข้อบังคับเพื่อความปลอดภัยในการทำหน้าที่ปฏิบัติการระบบจริง:

- Verify ก่อนประกาศเสร็จสิ้น: ห้ามอ้างอิงหรือกล่าวรายงานความสำเร็จโดยปราศจากการรันตรวจสอบจริงด้วยสายตาและเครื่องมือวิเคราะห์ (เช่น การตรวจจับ byte-for-byte บน follower ก่อนยืนยันว่า P2P Gossip ทำงานได้เรียบร้อยแล้ว)
- ทำสำรองก่อนแก้ไขเสมอ (Backup first): ทุกครั้งที่ต้องแก้ไขหรือตั้งค่าไฟล์คอนฟิกูเรชันในสภาพแวดล้อมจริง ให้ทำสำรองไฟล์พร้อมระบุหมายเลขเวลา (`.bak.<timestamp>`) เพื่อป้องกันภัยพิบัติข้อมูล

- **** patterns over intentions (รูปแบบพฤติกรรมเหนือเจตจำนง)****: รายงานเฉพาะสิ่งที่เกิดขึ้นในประวัติระบบและในผลการวิเคราะห์ทางวิศวกรรมจริงเท่านั้น ห้ามเดาหรือ narrate ข้อมูลเอาเองโดยปราศจากหลักฐานเชิงประจักษ์

7.3 คู่มือรายการตรวจสอบสุดท้ายสำหรับการเปิดเซน (Technical

Verification Checklist)

ก่อนเปิดใช้งานและซิงค์เครือข่ายบล็อกเชน Layer 2 (Follower Node) ให้ทำการตรวจสอบและตั้งค่าระบบตามเช็คลิสต์ทางเทคนิคดังนี้:

ก. รายการตรวจสอบฝั่ง libp2p (op-node ↔ op-node, เส้นทางซิงค์หลัก)

- □

1. ความสอดคล้องของ Genesis & Rollup Config

:

- ▶ รหัสแฮชบล็อกตั้งต้น (Chain Identity) ต้องมีความสอดคล้องต้องกันทั้ง 3 ทางอย่างสมบูรณ์แบบ ได้แก่:

- การเริ่มต้นฐานข้อมูลของ EL client (`geth init` / `reth init`)
- การตั้งค่า L2 Genesis Hash ในไฟล์คอนฟิกูเรชัน `rollup.json` (ฟิลด์ `genesis.l2.hash`)

- ค่าแฮชบล็อกที่ 0 (Block 0) บนเครือข่ายจริง (Live Block 0)

- ▶ คำเตือน: หากมีจุดใดจุดหนึ่งไม่ตรงกัน โหนด `op-node` จะทำการปฏิเสธ (Reject) และระงับการทำงานร่วมกันทันที

- □

2. การระบุเป้าหมายโหนดหลัก (Static Peer สำหรับ Follower)

:

- ▶ โหนดติดตามทั่วไป (Follower) จะต้องใส่ flag `--p2p.static` ชี้เข้าไปที่ Multiaddr ของ Sequencer (เช่น

```
--p2p.static=/ip4/141.11.156.4/tcp/9227/p2p/16Uiu2HAKzt25 ... )
```

หากละเว้น โหนดจะไม่สามารถทำ Peer discovery เพื่อซิงค์ unsafe blocks ในช่วงแรกได้

- □

3. กฎหมายลายมือชื่อของโหนดหลัก (Sequencer Signature Key)

:

- ▶ ผังโหนดผู้ผลิตบล็อก (Sequencer op-node) จะต้องมีการเปิดใช้งานและระบุ flag

```
--p2p.sequencer.key เสมอ เพื่อลงลายมือชื่อบล็อก (Sign block) และ
```

แพร่กระจายผ่าน P2P Gossip mesh

- ▶ บทเรียนจากปฏิบัติการจริง: หากขาดการระบุคีย์นี้ โหนดติดตาม (Followers) ในเครือข่ายทั้งหมดจะไม่สามารถตรวจสอบความปลอดภัยและจะไม่เชื่อมต่อ P2P gossip

- □

4. ความปลอดภัยของ Engine API ในโหนดตนเอง (Local JWT & RPC

Connection):

- ▶ ต้องมีการระบุความลับในการสตรีมคำสั่งสื่อสารระหว่าง `op-node` และ EL client ของตนเองอย่างถูกต้องผ่านคู่พารามิเตอร์ `--l2.jwt-secret` และ `--l2=authrpc` (พอร์ตเริ่มต้น 8551)

ข. รายการตรวจสอบฝั่ง EL devp2p (op-geth ↔ op-reth ตรงๆ, สำหรับ snap sync)

- □

5. การตั้งค่า P2P เพื่อการซิงค์ด่วน (Snap Sync Configuration)

:

- ▶ หากต้องการให้ไคลเอนต์ฝั่ง Execution (เช่น Go-Geth และ Rust-Reth) ทำการสื่อสารและซิงค์ข้อมูลสถานะด้วยตัวเองตรงๆ:
 - ต้องตั้งค่า enode/bootnodes ของอีกฝั่งที่เลเยอร์ EL
 - ต้องระบุค่าโควตาการรับเพื่อนสูงสุดผ่าน flag `--maxpeers` ให้มีค่ามากกว่า 0
 - ต้องตรวจสอบเลข Chain ID / Network ID ให้ตรงกัน
 - ต้องทำการตั้งค่า flag `--syncmode=execution-layer` บนโหนด `op-node` เพื่อให้มันอนุญาตและรอรับการซิงค์ประวัติผ่านกึ่ง EL แทนที่จะเริ่มดึงบล็อกจากจุดเริ่มต้น L1 ทันที

7.4 บทเรียนเพิ่มเติมจากการปฏิบัติการจริง (Operational Edge Cases & Tool Caveats)

จากการทดสอบ deploy จริงของสภาออราเคิลพบรายละเอียดและข้อผิดพลาดในการปฏิบัติการเพิ่มเติมที่ต้องระมัดระวังอย่างยิ่ง:

1. ข้อบังคับการระบุ Beacon API สำหรับ op-node เวอร์ชันใหม่: ในซอฟต์แวร์ `op-node` เวอร์ชันใหม่ๆ สัญญาฉันทามติกำหนดให้ต้องมีการดึงข้อมูล blob หรือสารสนเทศฉันทามติของ L1 ผ่านทาง Consensus Layer Beacon API ดังนั้น การรันคำสั่งโดยระบุเพียงแค่ L1 Execution RPC (`--l1`) จะทำให้เกิดข้อผิดพลาดและโปรแกรมปิดตัวลงทันที (Crash)
 - ข้อปฏิบัติ: ต้องมีการระบุ flag `--l1.beacon` ควบคู่ไปพร้อมกับ Beacon API endpoint ที่ถูกต้อง (เช่น โสสต์ของ Lighthouse/Prysm บน L1 Sepolia)
2. ปัญหาควบคุมตัวอักษรของคำสั่งผ่านเครื่องมือกรองกราฟฟิก (Control Characters over RTK Wrapper): การเรียกใช้ `curl` เพื่อตรวจสอบสถานะของโหนดผ่านเครื่องมือคอมเพรสเซอร์หรือพรีกซี (เช่น `rtk` wrapper) อาจทำให้เนื้อหาผลลัพธ์ (STDOUT)

มีการปนเปื้อนด้วยตัวอักษรควบคุมการแสดงผลและสี (Terminal Control / Color Escape Characters) ซึ่งส่งผลให้เมื่อนำผลลัพธ์ JSON ไปกรองหรือ parse ด้วยโปรแกรมอย่าง `jq` หรือ regex ตรงๆ จะเกิดความล้มเหลว (Parse Error)

- ข้อปฏิบัติ: หลีกเลี่ยงการ parse ข้อมูลดิบจากท่อสตรีมโดยตรงหากใช้ wrapper ให้บันทึกผลลัพธ์เป็นไฟล์ผ่านการ Redirect output (เช่น `curl ... > status.json`) แล้วค่อยเรียกโปรแกรมมาแกะวิเคราะห์จากไฟล์นั้นแทน หรือใช้ parameter พร็อกซีตรงที่ไม่มีการกรองรหัสสี

3. ความสำคัญของการตั้งค่าและการตรวจสอบ Default ของ `--l2.enginekind` (กับดัก

Weizen): ในโปรแกรม `op-node` เวอร์ชันใหม่ ค่าดีฟอลต์ (Default) ของ flag

`--l2.enginekind` ได้เปลี่ยนผ่านไปสู่ `reth` (แทนที่จะเป็น `geth` แบบเดิม)

- ผลกระทบ: หากผู้รันโนหนดทำการรัน `op-geth` โดยไม่ได้เจาะจงระบุ flag `--l2.enginekind=geth` ระบบจะใช้พฤติกรรมการเชื่อมต่อแบบ `reth` ทำให้เกิดพฤติกรรมที่ไม่เข้าคู่กัน (Behavior Mismatch) ซึ่งอาจส่งผลเสียในกรณีเกิด Edge cases ได้โดยไม่มี error ชัดแจ้ง
- ข้อปฏิบัติ: ห้ามทักท้วงเอาเองว่าค่าเริ่มต้นจะตรงกับซอฟต์แวร์ที่ใช้ ให้ตรวจสอบคำสั่งช่วยเหลือ (`op-node --help`) ของเวอร์ชันนั้นๆ เสมอ และเจาะจงระบุ flag ให้สอดคล้องกับ Execution Client ที่เลือกใช้งานจริง (เช่น `--l2.enginekind=geth` สำหรับ `op-geth` หรือ `--l2.enginekind=reth` สำหรับ `op-reth`)

4. ปัญหาสถานะฐานข้อมูลทับซ้อนเมื่อรันโนหนดหลายตัวบนโฮสต์เดียวกัน (กับดัก Tonk):

การเปิดใช้งานโนหนด `op-node` หลายตัวบนเซิร์ฟเวอร์หรือสภาพแวดล้อมเครื่องเดียวกัน หากรันคำสั่งจากโฟลเดอร์ทำงานเดียวกัน (Same Working Directory) หรือชี้

โฟลเดอร์เก็บข้อมูลร่วมกัน จะทำให้ไฟล์ฐานข้อมูลประวัติและเพื่อนบ้าน (`peerstore` และ `discovery db`) เกิดการเข้าถึงทับซ้อนและเขียนทับข้อมูลซึ่งกันและกัน

(Database Lock / Collision State)

- ข้อปฏิบัติ: ต้องรันคำสั่งโดยแยกโพลเดอร์ปฏิบัติการ (Working Directory - CWD) ของแต่ละโหนดออกจากกันโดยเด็ดขาด หรือแยกพารามิเตอร์ peerstore และ discovery database ให้เป็นเอกเทศจากกันอย่างชัดเจน

7.5 สรุปข้อพิสูจน์สถาปัตยกรรมแบบผสมไคลเอนต์ (Cross-Client Proof Matrix)

จากการร่วมมือวิจัยและทดสอบสลับซิงก์จริงระหว่าง `op-geth` และ `op-reth` ในเครือข่ายจำลอง สภาออราเคิลได้สรุปตารางข้อพิสูจน์ (Proof Matrix) เอาไว้เป็นบทเรียนดังนี้:

สิ่งที่พิสูจน์แล้ว (Proven) 

1. Cross-client op-node P2P peer connection works: โหนด `op-node` ที่ขับเคลื่อนด้วย `op-reth` (Rust) สามารถทำการเชื่อมต่อและสร้าง peer handshake ร่วมกับโหนด `op-node` ที่ขับเคลื่อนด้วย `op-geth` (Go) ได้อย่างถูกต้องสมบูรณ์แบบ
2. Client-diversity follower joins mesh successfully: โหนดติดตาม (Follower) ที่เป็น `op-reth` สามารถเข้าร่วมและเชื่อมต่อเข้าสู่ Gossip Mesh เดียวกันกับโหนดหลักที่เป็น `op-geth` ได้เป็นเนื้อเดียว
3. Chain consistency remains canonical: รหัสบล็อกที่ประมวลผลได้มีความเหมือนกันทุกไบต์ (Byte-for-Byte block hash identical) ข้ามเครื่องและข้ามชนิดไคลเอนต์ (ตรงตามค่าเฉลี่ย canonical genesis ของสภา)

สิ่งที่ยังไม่ได้รับการพิสูจน์เชิงประจักษ์ (Not Yet Proven) 

1. Cross-client unsafe block gossip delivery as primary source: ยังไม่มีข้อพิสูจน์หรือพยานหลักฐานประจักษ์ชัดว่าระบบได้ใช้ท่อนส่ง unsafe blocks gossip (A → B) เป็นช่องทางซิงก์หลักในการทำงานจริง เนื่องจากในสภาพแวดล้อมทดสอบ ตัวโหนดได้ทำการแกะชุดธุรกรรมย้อนหลังจาก L1 (L1 Derivation) ได้รวดเร็วและพร้อมๆ กันพอดี ทำให้

พฤติกรรมการพึ่งพา unsafe blocks gossip โดนบดบังไป (Masked) โดยหากต้องการพิสูจน์ข้อนี้อย่างชัดเจน จะต้องจำลองในสถานะที่โหนด follower ค้างหรือตามหลังการระเบิดบล็อกสดของ sequencer อยู่ระดับหนึ่งจนต้องดึง unsafe block gossip ข้ามโหนดมาใช้งานแทน

หากข้อมูลตามรายการด้านบนได้รับการลงลายมือชื่อตรวจสอบว่าผ่านพ้นทั้งหมด
เครือข่าย Layer 2 นั้นก็พร้อมที่จะให้บริการประมวลผลธุรกรรมแก่ชุมชนอย่างสมบูรณ์
แบบ
